iOS Transaction Guide

The **iOS Transaction Guide** provides a step-by-step walkthrough for implementing payment transactions using the TransactionManager class. It focuses on the following key areas:

- **Create the TransactionManager Object**: Define and configure merchant, terminal, and POS application objects required for transaction processing.
- **Prepare & Execute the Transaction**: Prepare transaction requests, and execute them using the appropriate methods.
- **Implement Real-Time Updates**: Implement the TransactionManagerDelegate to receive real-time updates on transaction progress, success, and failure.
- **Response Handling**: The onTransactionSucceeded and onTransactionFailed methods handle payment outcomes by confirming success with transaction details or reporting failure with status, terminal state, and error info.
- **Canceling an Ongoing Transaction:** Safely cancel an ongoing transaction.

Prerequisites

Before you continue, ensure you have the following prerequisites completed and ready:

- **Merchant Client Application Credentials and Access Token:** Ensure that merchant onboarding is complete and that you've received your client application credentials and access token before proceeding.
- **Access to Deluxe Developer Portal:** This portal provides additional resources, API documentation, and integration tools.

Note: *If you don't have a Deluxe Merchant Account or Access Token, simply visit Merchant Services Get Started and contact us today.*

Create the TransactionManager Object

Define and configure merchant, terminal, and POS application objects required for transaction processing.

Step 1: Create and Initialize the TransactionManager Object

Start by creating a TransactionManager instance using the relevant merchant, terminal, and POS application objects.

```
let transactionManager = TransactionManager(merchantInfo:
merchantInfo,
                          deviceInfo: terminalInfo,
                          merchantPosAppInfo: merchantPosAppInfo)
```

Note: *If the merchant account is configured to support surcharges, you must explicitly set the enableSurcharge flag to true. This ensures that the surcharge amount is calculated and applied correctly during the transaction.*

Step 2: initialize the TransactionManager

ON THIS PAGE

Prerequisites

Create the TransactionManager Object

Prepare & Execute the Transaction

Implement Real-Time Updates

Response Handling

Canceling an Ongoing Transaction

```
transactionManager?.initialize(delegate: self)
```

In the example above, the same class is assumed to conform to the TransactionManagerDelegate protocol.

How to determine if a terminal is supported for transactions

```
transactionManager?.isTerminalSupported(terminalInformation:
terminalInfo)
```

The **isTerminalSupported** method is used to determine whether a iOS device supports given terminal for payment. It checks the provided terminal information and returns a status indicating whether the device is capable of performing a payment using the given terminal.

TerminalSupportStatus: Enum that indicates different statuses here.

- **TerminalSupportStatus.supported**: Indicates that the device supports the given terminal.
- **TerminalSupportStatus.notSupported**: Indicates that the device does not support given terminal.

Prepare & Execute the Transaction

Step 1: Prepare Transaction Request

Create a TransactionRequest object tailored to the type of transaction.

Initialize an AmountInformation object with default values—such as amount, tip, cashback, fees, and tax—all set to zero. You can update these fields as needed based on your transaction requirements.

```
var amountInformation = AmountInformation(amount: 10.00,
                            adjustedAmount: 0.00,
                            tip: 0.00,
                            cashback: 0.00,
                            convenienceFee: 0.00,
                            serviceCharge: 0.00,
                            tax: 0.00,
                            tax: 0.00)
```

```
//This is an example of a SALE transaction request.
var transactionRequest = TransactionRequest( transactionType =
TransactionType.sale,
transactionSuperType = TransactionSuperType.credit,
transactionId = "",
stin = "",
allowPartialApproval = false,
originalTransactionType: TransactionType.sale,
amountInformation: amountInformation,
merchantCartItem: cartItems,
isManualKeyedIn = false,
cardData = nil,// Nil because its not a manual card entry sale
transaction
isTipEnabled = false, // It indicates if you opt for inline tip
additionalData = nil,
cashTransactionInformation = nil, // Nil because its not a cash/
tender sale transaction
```

Step 2: Perform Transaction

```
transactionManager?.performTransaction(transactionRequest:
transactionRequest)
```

Implement Real-Time Updates

Implement onTransactionProgressUpdate to receive various progress updates during transaction processing.

This delegate method is triggered whenever there's a progress update during transaction processing. Updates may relate to terminal connectivity or the transaction itself. The method can be called multiple times, providing the current transaction status, terminal status, a descriptive message, and any relevant data—along with the associated TerminalManager instance handling the interaction.

```
func onTransactionProgressUpdate(transactionStatus:
TransactionStatus,
                   terminalStatus: TerminalConnectionStatus,
                   description: String,
                   data: Any?,
                   terminalManager: TerminalManager?) {
// Here you will get Tranaction status and terminal status as well .
Based on this you can perform some action or change UI
}
```

The following data is passed into this method:

transactionStatus: Current status of the transaction - This will be always inProgress.

terminalStatus: Current terminal connection status or transaction status (e.g., connected, pairing, paymentGatewayStartedProcessing etc.).

description: A string that provides additional context about the current status. In some cases, this may be a plain message (e.g., "Waiting for card"), while in others it may be paired with structured data in the data field.

data: A generic object (Any) used to pass additional structured information relevant to the current status. For example:

- When **terminalStatus** is **pairingLedIndicatorsAvailable**, data contains raw LED status information from the TerminalManager object which handles the terminal connection.
- When **terminalStatus** is **bluetoothDiscoveryCompleted**, data includes a list of discovered bluetooth terminals.
- When **terminalStatus** is **waitingForAdditionalChargeConfirmation**, data contains the additional charge information such as amount and additionalChargeType.
- For other statuses, **data** might be null.

Transaction progress updates include:

TerminalConnectionStatus.pairingLedIndicatorsAvailable: This will come only if the transaction is requested on Ingenico Moby 5500 terminal and **data** field contain the different LED statuses and this has to be displayed in a UI for merchant to complete the pairing process.

seconds. When this happens, the transaction is immediately marked as failed, and the **onTransactionFailed()** callback is triggered.

TerminalConnectionStatus.paymentGatewayProcessingStarted: Indicates that the card data is collected and sent to payment gateway for processing.

TerminalConnectionStatus.paymentGatewayProcessingCompleted: Indicates that the transaction is completed.

TerminalConnectionStatus.waitingForInlineTip: This status indicates that the terminal has completed all card-related actions and is now waiting for the user to enter a tip amount before the transaction can proceed. This process, known as inline tipping, occurs prior to finalizing the transaction. During this stage, the TransactionManager will pause and wait for the tip input via the updateTransactionAmount(updatedAmount) method.

```
updateTransactionAmount(amountInfo, tenderAmount
//amountInfo class which contains all kind of amount incliding tip
tenderAmount: Double
//tenderAmount can used in case of CASH transaction
){}
```

TerminalConnectionStatus.waitingForAdditionalChargeConfirmation: This status indicates that an additional charge is found during the transaction processing and transaction manager is waiting for the merchant confirmation to proceed further. The data parameter will hold the object corresponding to the type of additional charge applied.

If the merchant confirms the additional charge then the following method call must be invoked to resume the transaction processing.

```
self.transactionManager?.confirmAdditionalCharge(isConfirmed)
```

- If **isConfirmed** is true, the transaction manager will resume the transaction that is currently on hold.
- If **isConfirmed** is false, the transaction will be cancelled with the status additionalChargeConfirmationDenied.

TerminalConnectionStatus.additionalChargeConfirmationDenied: This status indicates that the user declined the additional charge confirmation.

TerminalConnectionStatus.additionalChargeConfirmationFailedDueToTimedout: This status is triggered when the merchant does not respond to the additional confirmation within 120 seconds timeout.

In this case, the transaction will be cancelled automatically by the transaction manager.

TerminalConnectionStatus.additionalChargeConfirmationFailed: This indicates that the surcharge confirmation is failed due to any other error and the transaction will be cancelled by the Transaction Manager.

outcomes by confirming success with transaction details or reporting failure with status, terminal state, and error info.

Implement Transaction Success

The **onTransactionSucceeded(...)** method is triggered when a transaction has been successfully processed by the payment processor. It provides both the transaction status and a detailed response object containing all relevant transaction information. This method is typically used to confirm the successful payment to the user, generate or print receipts, and perform any post-transaction operations, such as updating the UI.

The **transactionStatus** confirms the completion of the transaction by the payment processor, while the **transactionResponse** includes the actual response from the payment processor, containing essential details such as the approval code, transaction ID, card type, processed amount, and receipt information.

```
func onTransactionSucceeded(transactionStatus: TransactionStatus,
              transactionResponse: TransactionResponse?) {
//Handle on transaction succeeded with transactionStatus and
transactionResponse
}
```

Transaction Failure Handling

The **onTransactionFailed(...)** method is triggered when a transaction could not be completed due to an error or interruption. This method provides the transaction status, the terminal's last known state, and an error message that describes the cause of the failure.

The **status** indicates the general failure of the overall transaction.

The **terminalStatus** helps identify the terminal's last known state at the time of failure (e.g., disconnected, card not presented), which could also be a reason for transaction failures. For example: bluetoothDeviceNotFound, pairingCancelled, pairingFailed.

The **error** string provides a description of the error that caused the transaction to fail.

```
func onTransactionFailed(status: PosSDK.TransactionStatus,
           terminalStatus: TerminalConnectionStatus,
           error: String,
           transactionResponse: TransactionResponse?) {
//Handle on transaction Failed with  status,terminalStatus,error
}
```

Canceling an Ongoing Transaction

Step 1: Cancel Transaction

Invoke the cancelTransaction() method to start the cancellation process and make sure to use the same Transaction Manager instance that has been used to initiate the original transaction that you trying to cancel.

This method will start the cancel process asynchronously and response will be send back to the Transaction Delegate as part of the TransactionManager object.

Step 2: Handle Cancel Transaction Callbacks

```
func onCancelTransactionSucceeded() {

}
func onCancelTransactionFailed(error: String, terminalStatus:
TerminalConnectionStatus) {

}
```

The following 2 callback methods will be notified with the cancel transaction responses.

onCancelTransactionSucceeded(): This method is invoked when a transaction cancellation request completes successfully. It indicates that the transaction was canceled without any errors.

onCancelTransactionFailed(error, terminalStatus): This method is called when a transaction cancellation attempt fails. It provides information about the failure through the error message and reports the current connection status of the terminal via terminalStatus.

Possible error scenarios represented by different terminal statuses are given below:

- **terminalConfigurationInProgress**: Indicates that the terminal is currently undergoing a configuration process. The cancellation request cannot be processed until the configuration is completed.
- **transactionIsProcessingWithGateway**: Indicates that the transaction is already being processed with the payment gateway. The terminal cannot cancel the transaction since it has moved past the cancellable stage.
- **transactionIsNotInProgress**: Indicates that there is no active transaction available to cancel.

Note: *Please refer to the Complete SDK Reference for additional details iOS SDK Reference Guide*

deluxe. connect

Payments API

Deluxe.com | Privacy Policy | Do Not Sell My Personal Information

Deluxe Corporation, 801 S Marquette Ave, Minneapolis, MN 55402.

© 2026 Deluxe Corporation